Prasanna Sanjay Kasar
Email: prasanna.kasar06@gmail.com

GSoC 2025 Exercises for SOFIE Projects: Enhancing Keras Parser and JAX/FLAX Integration

19th March 2025

Table of Contents

Overview

This document presents my solutions to the GSoC 2025 exercises for SOFIE, focused on enhancing the Keras Parser and JAX/FLAX integration. Below are the key links to solutions to the relevant exercises:

1. **Personal Development Branch**:
   GitHub - PrasannaKasar Root Branch:
   https://github.com/PrasannaKasar/root/tree/prasanna

2. **Keras/FLAX Task Solutions**:
   GitHub - Keras and FLAX task GSoC'25 Solution link:
   https://github.com/PrasannaKasar/root/tree/prasanna/tmva/sofie/Keras%20and%20Flax%

20tasks%20GSoC'25

3. **Preliminary Tasks**: https://docs.google.com/document/d/1TZWTqdAzkDGCTOEmeXiAMyxhBc5nM4bt8u_FHRi-gmY/edit?usp=sharing

These links include my personal development branch, the solutions for Keras/FLAX tasks, and documentation of preliminary tasks of SOFIE.

Preliminary Tasks Completed

Before addressing the tasks provided in this document, I have completed the preliminary exercises related to ROOT and TMVA as described in the official document. These tasks can be found here, and I have thoroughly explored and understood the necessary tools and concepts outlined in that document.

Exercise 3 – Play with the SOFIE Keras Parser and FLAX Library

Objective:

The goal of Exercise 3 was to explore SOFIE's Keras Parser and its capabilities to convert Keras models into its own Intermediate Representation (IR). Additionally, this exercise aimed to investigate the integration of FLAX NNX (Neural Networks for JAX) with SOFIE.

Approach:

1. **Keras Model Generation**: I first created a custom Keras model using TensorFlow/Keras and saved it as `CustomKerasModel.h5`. This step involved defining a simple yet representative neural network architecture in Keras, training it, and exporting the model.

2. **Model Parsing with SOFIE Keras Parser**: The saved model (`CustomKerasModel.h5`) was then parsed using the SOFIE Keras Parser (`SOFIE::PyKeras::Parse`). Using the `Parse_Keras_Model.C` macro, I converted the Keras model into the SOFIE Intermediate Representation. The parsing was executed through the root command line

with the `.x Parse_Keras_Model` command, which successfully translated the Keras model into SOFIE's format.

```
// This macro runs the SOFIE parser on a custom Keras model
// 'CustomKerasModel.h5'

using namespace TMVA::Experimental;


void Parse_Keras_Model(const char * modelFile = "CustomKerasModel.h5"){

    // check if the model file exists
    if (gSystem->AccessPathName(modelFile)) {
        Error("TMVA_SOFIE_RDataFrame", "Model file not found");
        return;
    }

    // parse the model
    SOFIE::RModel model = SOFIE::PyKeras::Parse(modelFile);

    TString modelHeaderFile = modelFile;
    modelHeaderFile.ReplaceAll(".h5",".hxx");

    //Generate the inference code
    model.Generate();
    model.OutputGenerated(std::string(modelHeaderFile));

    // copy include in the same directory
    std::cout << "Model header file is in the same directory" << std::endl;
}
```

(fig 2.1 Keras Model Parser macro)

3. **Header File Generation**: After parsing the model, I generated a header file `CustomKerasModel.hxx` that contains the model's architecture, layer details, and configuration in the SOFIE format.

Outcome:

```
1  //Code generated automatically by TMVA for Inference of Model file [CustomKerasModel.h5] at [Sun Mar 16 17:42:35 2025]
2
3  #ifndef ROOT_TMVA_SOFIE_CUSTOMKERASMODEL
4  #define ROOT_TMVA_SOFIE_CUSTOMKERASMODEL
5
6  #include <algorithm>
7  #include <cmath>
8  #include <vector>
9  #include "TMVA/SOFIE_common.hxx"
10 #include <fstream>
11
12 namespace TMVA_SOFIE_CustomKerasModel{
13 namespace BLAS{
14    extern "C" void sgemv_(const char * trans, const int * m, const int * n, const float * alpha, const float * A,
15                           const int * lda, const float * X, const int * incx, const float * beta, const float * Y, const int * incy);
16    extern "C" void sgemm_(const char * transa, const char * transb, const int * m, const int * n, const int * k,
17                           const float * alpha, const float * A, const int * lda, const float * B, const int * ldb,
18                           const float * beta, float * C, const int * ldc);
19 }//BLAS
20 struct Session {
21 // initialized tensors
22 std::vector<float> fTensor_dense_4kernel0 = std::vector<float>(32);
23 float * tensor_dense_4kernel0 = fTensor_dense_4kernel0.data();
24 std::vector<float> fTensor_dense_3bias0 = std::vector<float>(8);
25 float * tensor_dense_3bias0 = fTensor_dense_3bias0.data();
26 std::vector<float> fTensor_dense_3kernel0 = std::vector<float>(128);
27 float * tensor_dense_3kernel0 = fTensor_dense_3kernel0.data();
28 std::vector<float> fTensor_dense_2kernel0 = std::vector<float>(512);
29 float * tensor_dense_2kernel0 = fTensor_dense_2kernel0.data();
30 std::vector<float> fTensor_dense_1bias0 = std::vector<float>(32);
31 float * tensor_dense_1bias0 = fTensor_dense_1bias0.data();
32 std::vector<float> fTensor_dense_4bias0 = std::vector<float>(4);
33 float * tensor_dense_4bias0 = fTensor_dense_4bias0.data();
34 std::vector<float> fTensor_dense_1kernel0 = std::vector<float>(2048);
35 float * tensor_dense_1kernel0 = fTensor_dense_1kernel0.data();
36 std::vector<float> fTensor_densebias0 = std::vector<float>(64);
37 float * tensor_densebias0 = fTensor_densebias0.data();
38 std::vector<float> fTensor_dense_2bias0 = std::vector<float>(16);
39 float * tensor_dense_2bias0 = fTensor_dense_2bias0.data();
40 std::vector<float> fTensor_densekernel0 = std::vector<float>(4096);
41 float * tensor_densekernel0 = fTensor_densekernel0.data();
42
43 //--- Allocating session memory pool to be used for allocating intermediate tensors
44 char* fIntermediateMemoryPool = new char[976];
```

(fig 2.2 First 44 lines of generated CustomKerasModel header file)

I have successfully explored the SOFIE Keras Parser, generating the required header file (`CustomKerasModel.hxx`) for further use. The parser accurately translated the Keras model, including its layer structure and associated configurations, into SOFIE's IR format.

Exercise 4 – Implement a Parsing Function

Objective:

The goal of Exercise 4 was to implement a function in Python that takes an in-memory FLAX model object and returns its configuration as a dictionary. This task required me to design a parsing function that can handle different FLAX model architectures and extract the relevant configuration details, including nested layers.

Approach:

1. **Model Definition**: I began by defining a FLAX model using the `flax.nnx` API, creating a simple neural network architecture with multiple layers. The model included both top-level layers (such as `Dense` layers) and nested layers (e.g., an encoder layer with sub-layers such as `linear1` and `linear2`).

```python
import flax.linen as nn

# Define a simple FLAX model with nested layers
class Encoder(nn.Module):
    def setup(self):
        self.linear1 = nn.Dense(128)
        self.linear2 = nn.Dense(64)

    def __call__(self, x):
        x = self.linear1(x)
        return self.linear2(x)

class MainModel(nn.Module):
    def setup(self):
        self.encoder = Encoder()

    def __call__(self, x):
        return self.encoder(x)

# Instantiate the model
model = MainModel()
```

(fig 3.1 Example of nested layered model in Flax)

2. **Parsing Function Implementation**: I then implemented the `get_model_config` function, which recursively parses the FLAX model and extracts the following configuration details:
    - **Model Name**: The name of the model class.
    - **Layer Information**: Information about each layer, including the layer name, class name, input/output shape, number of parameters, and data type.
    - **Total Number of Parameters**: The total number of parameters across the entire model.
3. The function is designed to handle nested layers by recursively processing each layer and sub-layer, ensuring no component of the model is missed.

4. **Testing**: I tested the function using various FLAX models, including models with nested layers. The function correctly returned the model configuration, including details about nested sub-layers.

(fig 3.3 get_model_config function)

```python
def get_model_config(model: nnx.Module) -> dict:
    '''
    Extracts the configuration of a Flax model recursively, so that any nested layers are not missed.

    Input:
        model: Flax model of type nnx.Module

    Output:
        A dictionary containing the model configuration, including:
        - Model name
        - Layer information (name, type, input/output dimensions, number of parameters, data type)
        - Total number of parameters in the model
    '''
    # Initialize a dictionary to store the overall model configuration
    model_config = {}

    # Store the model's class name (i.e., model type)
    model_config['Model Name'] = model.__class__.__name__

    # Initialize a dictionary to store detailed layer-wise information
    model_config['layer information'] = {}

    # Initialize a list to store the number of parameters used in every layer
    layer_param_counts = []

    # Helper function to recursively gather configuration data from the layers of the model
    def helper(layer_dict, layer_obj):
        '''
        Recursive function to extract configuration for each layer and its parameters

        Input:
            layer_dict: Dictionary to store layer information (will be passed recursively)
            layer_obj: The current layer object to extract information from
        '''

        # Iterate over the children (layers) of the current layer object
        for layer_name, layer_params in layer_obj.iter_children():

            # Initialize a new dictionary entry for each layer in the main dictionary
            layer_dict[layer_name] = {}

            # Store the class name of the current layer
            layer_dict[layer_name]['class'] = layer_params.__class__.__name__

            # If the current layer has nested layers, recursively extract information from those
            if any(layer_params.iter_children()):
                # If nested, initialize a dictionary for the nested layers
                layer_dict[layer_name]['layers'] = {}
                # Recursively call helper for nested layers
                helper(layer_dict[layer_name]['layers'], layer_params)
            else:
                # For layers without further nested layers, iterate over the parameters of the layer
                for param_type, param in vars(layer_params).items():

                    # Consider only parameters of type 'Param' with a 2D shape (weights/kernels)
                    if param.__class__.__name__ == 'Param' and param.value.ndim == 2:
                        # Extract the shape of the parameter (input_dim, output_dim)
                        param_shape = param.value.shape

                        # Store the input/output shape of the layer
                        layer_dict[layer_name]['shape'] = {
                            'input shape': (None, param_shape[0]),
                            'output shape': (None, param_shape[1])
                        }

                        # Calculate the total number of parameters for this layer
                        # Formula: (input_dim * output_dim) + output_dim (for the bias)
                        layer_param_count = param_shape[0] * param_shape[1] + param_shape[1]

                        # Append the number of parameters of this layer to the list
                        layer_param_counts.append(layer_param_count)

                        layer_dict[layer_name]['number of parameters'] = layer_param_count

                        # Store the data type of the parameters in the layer
                        layer_dict[layer_name]['dtype'] = str(param.value.dtype)

    # Start the recursion by passing the model's base layer information dictionary and model object
    helper(model_config['layer information'], model)
    # Calculate the total number of parameters used in the model
    total_number_of_params = sum(layer_param_counts)
    # Store the total number of parameters in the model configuration
    model_config['Total number of parameters'] = total_number_of_params      You, 43 minutes ago • Fl
    # Return the final model configuration
    return model_config
```

Outcome:

I successfully implemented the parsing function, which accurately extracts the configuration of FLAX models. The function is robust and can handle complex model architectures with nested layers, returning the desired configuration in dictionary form.

```
from pprint import pprint

# Print the model configuration
model_config = get_model_config(model)
pprint(model_config)
```

```
{'Model Name': 'MLP',
 'Total number of parameters': 97,
 'layer information': {'linear1': {'class': 'Linear',
                                   'dtype': 'float32',
                                   'number of parameters': 64,
                                   'shape': {'input shape': (None, 1),
                                             'output shape': (None, 32)}},
                       'linear2': {'class': 'Linear',
                                   'dtype': 'float32',
                                   'number of parameters': 33,
                                   'shape': {'input shape': (None, 32),
                                             'output shape': (None, 1)}}}}
```

(fig 3.3 Output of Model definition when parsed in get_model_config())

```
{'Model Name': 'VAE',
 'layer information': {'decoder': {'class': 'Decoder',
                                   'layers': {'linear1': {'class': 'Linear',
                                                          'dtype': 'float32',
                                                          'number of parameters': 8448,
                                                          'shape': {'input shape': (None,
                                                                                    32),
                                                                    'output shape': (None,
                                                                                     256)}},
                                              'linear2': {'class': 'Linear',
                                                          'dtype': 'float32',
                                                          'number of parameters': 201488,
                                                          'shape': {'input shape': (None,
                                                                                    256),
                                                                    'output shape': (None,
                                                                                     784)}}}},
                       'encoder': {'class': 'Encoder',
                                   'layers': {'linear1': {'class': 'Linear',
                                                          'dtype': 'float32',
                                                          'number of parameters': 200960,
                                                          'shape': {'input shape': (None,
                                                                                    784),
                                                                    'output shape': (None,
                                                                                     256)}},
                                              'linear_mean': {'class': 'Linear',
                                                              'dtype': 'float32',
                                                              'number of parameters': 8224,
                                                              'shape': {'input shape': (None,
                                                                                        256),
                                                                        'output shape': (None,
                                                                                         32)}},
                                              'linear_std': {'class': 'Linear',
                                                             'dtype': 'float32',
                                                             'number of parameters': 8224,
                                                             'shape': {'input shape': (None,
                                                                                       256),
                                                                       'output shape': (None,
                                                                                        32)}}}}}}
```

(fig 3.4 Output of sample model with nested layers)